Data Models

RDF and Knowledge Graphs

Tarapong Sreenuch

8 February 2026

克明峻德，格物致知

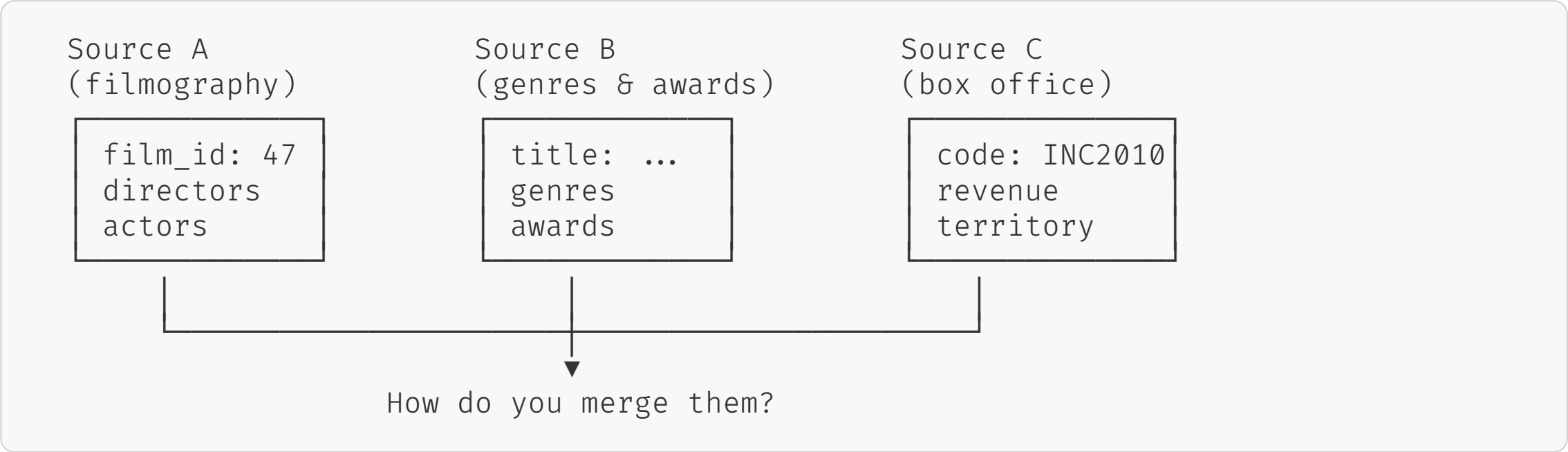
Building On — Topics 2, 5, 6, and 7

- **Topic 2:** Primary keys as unique identifiers — within *one* database
- **Topic 5:** Engine rejects — DDL constraints block invalid data at write time
- **Topic 6:** Document model accepts any shape within one collection (schema-on-read)
- **Topic 7:** Validator rejects — XSD checks documents at a trust boundary

Every model so far works inside a boundary — one database, one application, one trust boundary. What happens when data from **independent sources** needs to combine into a single queryable whole?

Three Sources, One Question

A film research team collects metadata from three independent databases. Each has its own schema, its own identifiers, and its own maintenance schedule.



Three things go wrong.

Failure 1 — Strings vs Things

A film research team merges metadata from two databases. They join on film titles.

```
Database A                          Database B

┌──────────────────────────┐        ┌──────────────────────────┐
│ "Crash" — 2004 drama     │        │ "Crash" — 1996           │
│ Won Best Picture         │        │ Cronenberg film          │
└──────────────────────────┘        └──────────────────────────┘
            │                                  │
            └──────────── "Crash" ─────────────┘
                       Same string.
                  Different films.
```

The team adds release year as a disambiguator: `("Crash", 2004)` vs `("Crash", 1996)`. A third source lists the 2004 film's year as `2005` (international release). The composite key disagrees. The merge breaks again.

Cost: 15 hours cleaning up false positives.

Failure 2 — No Shared Identifiers

Three sources describe the same 200 films. Each uses its own identifier scheme:

Source	Identifier for Inception
Source A	`film_id = 47` (auto-increment)
Source B	`slug = "inception-2010"`
Source C	`film_code = "INC2010"` (proprietary)

Same film, three IDs, no overlap. The team builds a **manual mapping table** — 600 rows, one per film-source pair.

One typo silently drops a film from all cross-source results. The mapping table is as fragile as the problem it was supposed to fix.

Failure 3 — No Vocabulary Contract

Three sources contribute edges about directors. Each invents its own predicate:

Source	Predicate	Direction	Meaning
A	`directedBy`	Film → Person	"this film was directed by..."
B	`director`	Film → Person	"this film's director is..."
C	`dir`	Person → Film	"this person directed..."

Sources A and B agree on direction (Film → Person) but use different names. Source C uses the same relationship but **reversed direction** (Person → Film). A query following `directedBy` edges finds Source A's data but misses B and C entirely.

Cost: Three published analyses retracted because the team didn't discover the inconsistency until after release.

The Pattern Across Three Failures

Failure	Root cause	What's missing
Strings vs things	Display names used as identifiers	Global, unambiguous identifiers
No shared identifiers	Local IDs with no cross-source scope	A shared identifier scheme
No vocabulary contract	Predicates invented independently	A formal vocabulary contract

Every failure has the same shape: **independent sources, no shared infrastructure for identity or vocabulary**.

> **Predict:** What kind of identifier could two independent teams adopt *without* coordinating their internal schemas — and still have their data merge automatically?

URIs: Identifiers That Work Across Sources

A **Uniform Resource Identifier (URI)** combines a namespace with a local name to produce a globally unique string:

```
http://example.org/movie#film_tt1375666
└──────── namespace ────────┘└─ local name ─┘
```

- **Namespace** scopes the identifier (no collisions across organisations)
- **Local name** identifies the entity within that scope (IMDB's `tt` code)

Two independent teams that adopt the same URI pattern (`ex:film_tt1375666` for Inception) have their data merge automatically — same URI, same node, edges connect. No mapping table.

> Therefore, the engineer must **build a shared identity and vocabulary layer** — global identifiers (URIs), atomic facts (triples), an entity-vs-literal distinction, a vocabulary contract (ontology), and traversal across source boundaries — not discover after deployment that three sources cannot merge.

Predict: URI Misalignment

Two sources both have data about Inception. Source A uses `ex:film_tt1375666`. Source B uses `ex:Inception_2010`.

After merging both sources into the same graph, a user traverses from Nolan → directed → ??? → hasGenre.

> **Predict:** Do the edges connect? Will the traversal reach Source B's genre triples?

> **No.** The graph has two disconnected nodes — `ex:film_tt1375666` (with Source A's edges) and `ex:Inception_2010` (with Source B's edges). Different URIs mean different nodes, even when both refer to the same real-world film. The fix is an equivalence triple (`owl:sameAs`), but that's manual reconciliation — exactly the mapping-table problem URIs were supposed to eliminate. URI *alignment* upfront is always cheaper than URI *reconciliation* after the fact.

Five Capabilities the Failures Demand

#	Capability	Which failure it prevents
1	**Global identifiers** (URIs)	Failure 1 (strings vs things) + Failure 2 (no shared IDs)
2	**Atomic facts** (triples: subject–predicate–object)	All three — a universal unit for cross-source statements
3	**Entity vs literal distinction**	Failure 1 — URIs are traversable, strings are dead ends
4	**Vocabulary contract** (ontology)	Failure 3 — shared predicate names, types, and directions
5	**Traversal across source boundaries**	All three — the integration payoff

The Resource Description Framework (RDF) provides all five by design.

The Triple: RDF's Atomic Fact

Every fact in RDF is a **triple**: subject–predicate–object.

```
┌──────────────┐    ┌──────────────────┐    ┌──────────────┐
│  Subject     │ ──→ │  Predicate       │ ──→ │  Object      │
│  (entity)    │    │  (relationship)  │    │  (entity or  │
│              │    │                  │    │   literal)   │
└──────────────┘    └──────────────────┘    └──────────────┘
```

Triple	Subject	Predicate	Object
"Nolan directed Inception"	`ex:person_nm0634240`	`ex:directed`	`ex:film_tt1375666`
"Inception's title is 'Inception'"	`ex:film_tt1375666`	`ex:hasTitle`	`"Inception"`

The first triple connects two URIs — both are traversable nodes. The second triple connects a URI to a **literal** (a plain value). Literals are leaf nodes — you can land on them, but you cannot traverse *from* them.

Entity vs Literal: Why It Matters

```
URI node (traversable)                Literal (dead end)

ex:film_tt1375666                     "Inception"
     │                                     │
     ├──hasGenre──→ ex:genre_scifi         (no outgoing edges)
     ├──wonAward──→ ex:award_bestpicture
     └──directed←── ex:person_nm0634240
```

- **URI** → a node with outgoing edges. You can follow it to genres, awards, directors.
- **Literal** → a leaf. The string `"Inception"` has no edges. It is display data, not an identity.

If a genre is stored as the literal `"Science Fiction"` instead of the URI `ex:genre_scifi`, the traversal "all films in this genre" breaks — the literal and the URI are different nodes.

RDF Vocabulary: Structure

Term	Definition
URI	Globally unique identifier for an entity (namespace + local name)
Triple	Atomic fact: subject–predicate–object
Subject	The entity the fact is about (always a URI)
Predicate	The relationship or property (always a URI)
Object	The target — a URI (entity) or a literal (value)
Literal	A plain value (string, integer) — a leaf node with no outgoing edges

RDF Vocabulary: Contracts and Assumptions

Term	Definition
Ontology	A formal vocabulary contract: classes, properties, domains, ranges
Object property	A predicate connecting two entities (URI → URI)
Datatype property	A predicate connecting an entity to a literal (URI → value)
Open-world assumption	Absence means *unknown*, not *false*

RDF Serialisation Formats

An RDF graph is abstract — triples need a concrete **serialisation format** to be stored or transmitted. The simplest is **N-Triples**: one triple per line, full URIs in angle brackets, literals in quotes.

```
<http://example.org/movie#person_nm0634240> <http://example.org/movie#directed> <http://example.org/movie#film_tt1375666> .
<http://example.org/movie#film_tt1375666>   <http://example.org/movie#hasTitle> "Inception" .
```

Unambiguous and trivially parseable — but extremely verbose. The full URI is repeated on every line. In a file with thousands of triples, the repeated namespaces dominate the file and obscure the data.

Other Serialisation Formats

JSON-LD — triples in a JSON structure. `@context` aliases namespaces, `@id` identifies subjects:

```
{
  "@context": { "ex": "http://example.org/movie#" },
  "@graph": [
    { "@id": "ex:person_nm0634240",
      "ex:directed": { "@id": "ex:film_tt1375666" } },
    { "@id": "ex:film_tt1375666",
      "ex:hasTitle": "Inception" }
  ]
}
```

RDF/XML — the oldest format. Wraps triples in XML elements:

```
<rdf:RDF xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"
         xmlns:ex="http://example.org/movie#">
  <rdf:Description rdf:about="http://example.org/movie#person_nm0634240">
    <ex:directed rdf:resource="http://example.org/movie#film_tt1375666"/>
  </rdf:Description>
</rdf:RDF>
```

JSON-LD suits web APIs. RDF/XML suits legacy toolchains. Both encode the same abstract graph.

Other Serialisation Formats

Turtle is a strict superset of N-Triples — every valid N-Triples file is valid Turtle. Turtle adds `@prefix` to eliminate repeated namespaces:

```
@prefix ex: <http://example.org/movie#> .

ex:person_nm0634240  ex:directed  ex:film_tt1375666 .
ex:film_tt1375666    ex:hasTitle  "Inception" .
```

Same two triples as the N-Triples version — far less noise.

Turtle also supports `@base` for a default namespace and three shortcuts (`;` `,` `.`) covered next. This chapter uses Turtle exclusively — it's the standard for hand-authored RDF data.

Turtle: Writing Triples

Turtle (Terse RDF Triple Language) is the syntax for writing triples. Prefixes shorten URIs:

```
@prefix ex: <http://example.org/movie#> .

ex:person_nm0634240   ex:directed   ex:film_tt1375666 .
```

This is one triple: Nolan (`ex:person_nm0634240`) directed Inception (`ex:film_tt1375666`).

Two shortcuts reduce repetition:

Shortcut	Meaning	Delimiter
`;` (Semicolon)	Same subject, different predicate	Add another predicate–object pair
`,` (Comma)	Same subject and predicate, different object	Add another object

Turtle Shortcuts in Action

```
ex:film_tt1375666   rdf:type       ex:Film ;
                    ex:hasTitle    "Inception" ;
                    ex:releaseYear "2010"^^xsd:integer .
```

Three triples, one subject (`ex:film_tt1375666`). The semicolons add new predicate–object pairs.

```
ex:person_nm1950086   a          ex:Director , ex:Actor ;
                      ex:hasName "Greta Gerwig" .
```

Three triples. The keyword `a` is Turtle shorthand for `rdf:type`. The comma after `ex:Director` adds a second type object — Gerwig is both Director and Actor.

Blank Nodes: Anonymous Intermediate Entities

Not every node needs a global URI. A **blank node** bundles related properties without a global identity — useful for intermediate entities that no other source needs to reference.

```
ex:film_tt1375666 ex:hasRating [ ex:platform "Rotten Tomatoes" ;
                                 ex:score    "8.5"^^xsd:decimal ] ,
                               [ ex:platform "IMDb" ;
                                 ex:score    "8.8"^^xsd:decimal ] .
```

The `[ ]` brackets create anonymous inline nodes. Equivalent labelled form: `_:r1`, `_:r2` (document-scoped only).

When to use blank nodes vs URIs: If another source might reference the entity (e.g., the Performance node in Slide 31), assign a URI. If the entity is local and disposable (e.g., a rating snapshot), a blank node suffices.

A blank node is a row with no primary key — it holds data but no foreign key from another table can point to it.

Ontology and Data: A Minimal Example

Before the full movie ontology, a two-class subset shows how ontology declarations and data instances relate:

```
ex:Film      a  owl:Class .
ex:Director  a  owl:Class .

ex:directed      a  owl:ObjectProperty ;
                 rdfs:domain  ex:Director ;
                 rdfs:range   ex:Film .

ex:releaseYear   a  owl:DatatypeProperty ;
                 rdfs:domain  ex:Film ;
                 rdfs:range   xsd:integer .
```

The keyword `a` is Turtle shorthand for `rdf:type` — it reads as "is a." The ontology and data can reside in the same file or separate files.

Data instances that conform to the contract:

```
ex:person_nm0634240  a  ex:Director ;
                     ex:hasName  "Christopher Nolan" .

ex:film_tt1375666    a  ex:Film ;
                     ex:hasTitle    "Inception" ;
                     ex:releaseYear "2010"^^xsd:integer .

ex:person_nm0634240  ex:directed  ex:film_tt1375666 .
```

The Movie Ontology: Object Properties

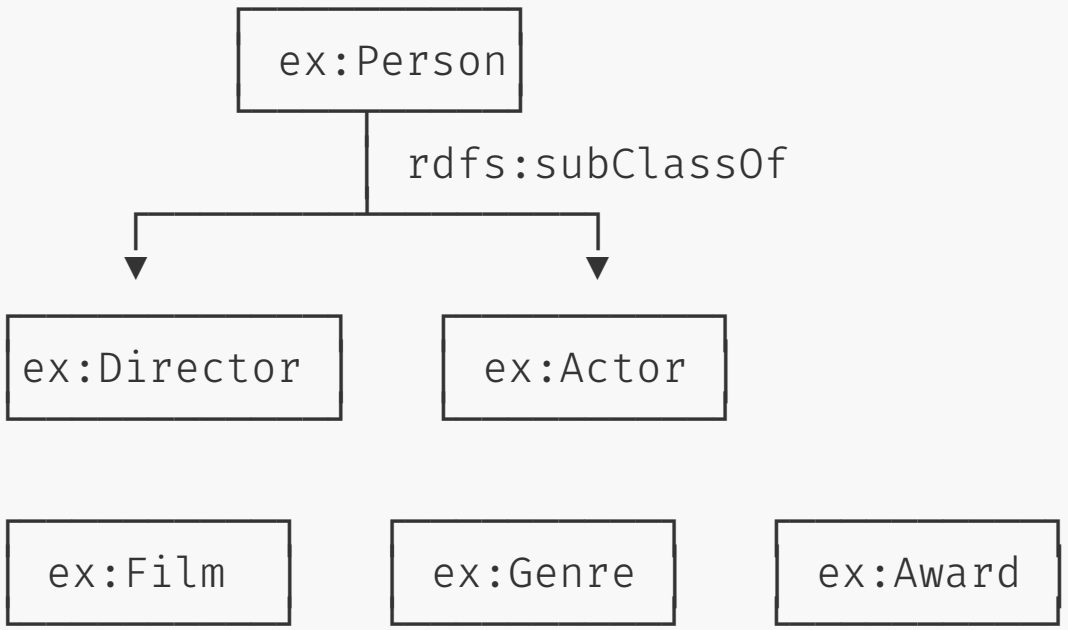
Object properties connect entities to entities (URI → URI):

Property	Domain	Range	Meaning
`ex:directed`	Person	Film	This person directed this film
`ex:actedIn`	Person	Film	This person acted in this film
`ex:hasGenre`	Film	Genre	This film belongs to this genre
`ex:wonAward`	Film	Award	This film won this award
`ex:sequelOf`	Film	Film	This film is a sequel of that film

Each row is a **property card** — the same structured extraction pattern as Topic 7's element contract cards. Domain tells you what type the subject must be; range tells you what type the object must be.

The Movie Ontology: Classes

The ontology declares six classes and two subclass relationships:



Director and Actor inherit from Person via `rdfs:subClassOf`. Any property with domain `ex:Person` accepts Directors and Actors in the subject position — subclass satisfies superclass.

The Movie Ontology: Datatype Properties

Datatype properties connect entities to literal values (URI → value):

Property	Domain	Range	Meaning
`ex:hasTitle`	Film	`xsd:string`	The film's display title
`ex:hasName`	(any)	`xsd:string`	The entity's display name
`ex:releaseYear`	Film	`xsd:integer`	The film's release year
`ex:runtime`	Film	`xsd:integer`	The film's runtime in minutes

Notice: `ex:hasName` has **no domain constraint** — it can be used on any entity (persons, genres, awards). `ex:hasTitle` is restricted to films. To resolve a URI to a human-readable name, you must know which property to follow — which means knowing the entity's type first.

Predict: The ontology declares `ex:runtime` with domain Film and range integer. None of the nine films in the seed data include a runtime triple. Under the open-world assumption, does this mean the films have no runtime, or that the runtime is unknown?

Open-World vs Closed-World

The runtime is **unknown**, not absent.

Assumption	Model	Absent data means	Example
Closed-world	Relational (Topic 2–5)	The fact is *false*	No row for Murphy in Villeneuve's filmography → Murphy did not act in a Villeneuve film
Open-world	RDF (this chapter)	The fact is *unknown*	No runtime triple for Inception → the graph doesn't have that fact *yet*

An empty traversal result means "the graph does not contain this path" — not "this relationship does not exist in the world." Another source might add the triple next week.

The Enforcement Spectrum: Now Complete

Topic	Contract	Enforcement	Consequence of violation
5 (Relational)	DDL + constraints	**Engine rejects** at write time	Row never enters the database
7 (XML)	XSD	**Validator rejects** at boundary	Document never reaches application
8 (RDF)	Ontology	**Nothing rejects**	Triple enters the graph; downstream traversals may produce wrong results

Predict: Someone adds `ex:genre_scifi ex:directed ex:genre_action` to the movie graph. The ontology says `ex:directed` has domain Person and range Film. What happens?

Nothing Rejects: The Triple Store Accepts It

The triple `ex:genre_scifi ex:directed ex:genre_action` enters the graph. A Genre "directing" a Genre — structurally nonsensical but syntactically valid. No error message.

Why? There is no single gatekeeper — each source contributes triples independently. Enforcement would require either a central authority (negating source independence) or universal validators (requiring coordination that may not exist).

The ontology says: "Here is what the properties mean. Follow this vocabulary and your data integrates correctly. Don't follow it — and your data integrates *incorrectly*, but it still integrates."

Cost: Invalid triples enter the graph silently. Correctness is a social contract, not a technical guarantee.

Why URIs Beat Local IDs

Problem with local IDs	How URIs solve it
No uniqueness guarantee — Source A's `film_id = 47` and Source B's `film_id = 47` may be different films	**Global uniqueness** by convention: `ex:film_tt1375666` cannot collide with another namespace's identifier
No shared scheme — integers, slugs, proprietary codes don't match syntactically	**Shared scheme** by agreement: all sources adopt the same URI pattern (IMDB `tt` codes)
Mapping is manual and fragile — one typo drops an entity	**No mapping table**: the URI *is* the mapping. Same URI → same node → edges connect

Cost accepted: Requires upfront alignment agreement across sources.

XML Nesting → RDF Triples

In Topic 7's XML, Chalamet is an `<actor>` element nested inside `<film>` — his identity is *positional*. In RDF, he is a URI that exists independently of any document.

```
<film id="tt1160419" title="Dune" year="2021">
  <cast>
    <actor id="nm3154303" name="Timothée Chalamet"
           role="Paul Atreides"/>
  </cast>
</film>
```

Three design decisions arise when converting this to triples.

> **Predict:** In the XML, Chalamet exists only inside Dune's `<cast>` element. In RDF, he is the URI `ex:person_nm3154303`. Can a second source add facts about Chalamet without modifying Source A's data?

Three Conversion Decisions

Yes — a second source writes `ex:person_nm3154303 ex:actedIn ex:film_tt2543164` and the graph absorbs it. Source A is untouched. That's global addressability.

XML construct	RDF equivalent	Design decision
`id="tt1160419"`	URI: `ex:film_tt1160419`	Attribute value becomes globally unique identifier
`title="Dune"`	Literal: `ex:hasTitle "Dune"`	Display data becomes a datatype property
`<cast>` → `<actor>` nesting	`ex:actedIn` edge	Containment becomes a named typed edge
`role="Paul Atreides"`	Performance intermediate node	Relationship attribute needs an associative entity
Sibling order of `<actor>` elements	Lost (unordered triples)	Order not preserved — acceptable when meaningless

General principle: If you'll ever want to traverse *to* or *from* something, it needs a URI. If it's just display data, a literal is fine.

The Performance Node: Relationship Attributes in RDF

The `role="Paul Atreides"` attribute belongs to neither the actor nor the film — it belongs to the *relationship* between them. Topic 2's junction table (OrderLine with `quantity` and `unit_price`) solved the same problem. In RDF, you need an **intermediate node**:

```
ex:performance_001   rdf:type       ex:Performance ;
                     ex:hasActor    ex:person_nm3154303 ;
                     ex:hasFilm     ex:film_tt1160419 ;
                     ex:characterName "Paul Atreides" .
```

More verbose than a direct `ex:actedIn` edge, but the triple model has no other way to attach attributes to a relationship — edges cannot carry properties.

Topic 10's property graphs offer a simpler answer: an `ACTED_IN` edge carries a `role` property directly.

Design Choices: Trade-Off Summary

Design Choice	Problem It Solves	Cost Accepted
URIs instead of local IDs	Global uniqueness; no mapping tables	Requires upfront alignment agreement
Ontology describes, doesn't enforce	No single gatekeeper needed; sources stay independent	Invalid triples enter the graph silently
Graph traversals over join chains	Exploratory, multi-hop queries without pre-declared paths	Weaker transactional guarantees than relational
XML nesting → flat triples	Global addressability; trivial cross-source linking	Element ordering and containment hierarchy lost

Consider: "Actors who appeared in films directed by both Nolan and Villeneuve." In a relational schema, that requires a multi-table join chain with pre-declared foreign keys. In a graph, the traversal path emerges from the data — `directed` edges to films, `actedIn` edges reversed, intersect. No model is universally better — the choice follows the workload.

Two Sources, Shared URIs

Source A — filmography data (directors, actors, films, directed/actedIn edges):

```
@prefix rdf:  <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix xsd:  <http://www.w3.org/2001/XMLSchema#> .
@prefix ex:   <http://example.org/movie#> .

ex:person_nm0634240   rdf:type     ex:Director ;
                      ex:hasName   "Christopher Nolan" .

ex:person_nm1950086   rdf:type     ex:Director , ex:Actor ;
                      ex:hasName   "Greta Gerwig" .

ex:film_tt1375666     rdf:type     ex:Film ;
                      ex:hasTitle  "Inception" ;
                      ex:releaseYear "2010"^^xsd:integer .

ex:film_tt0468569     rdf:type     ex:Film ;
                      ex:hasTitle  "The Dark Knight" ;
                      ex:releaseYear "2008"^^xsd:integer ;
                      ex:sequelOf    ex:film_tt0372784 .

...
```

```
...

ex:film_tt15398776    rdf:type     ex:Film ;
                      ex:hasTitle  "Oppenheimer" ;
                      ex:releaseYear "2023"^^xsd:integer .

ex:person_nm0614165   rdf:type     ex:Actor ;
                      ex:hasName   "Cillian Murphy" .

ex:person_nm0634240   ex:directed  ex:film_tt1375666 .
ex:person_nm0634240   ex:directed  ex:film_tt0468569 .

ex:person_nm0614165   ex:actedIn   ex:film_tt1375666 .
ex:person_nm0614165   ex:actedIn   ex:film_tt15398776 .
```

Source A has 3 directors, 9 films, 3 actors — 58 triples total. Complete `directed` and `actedIn` edges:

Actor	actedIn → Films
Murphy	The Dark Knight, Inception, Oppenheimer
Chalamet	Dune, Lady Bird
Robbie	Barbie, Oppenheimer

Director	directed → Films
Nolan	Inception, Interstellar, Oppenheimer, Batman Begins, The Dark Knight, The Dark Knight Rises
Villeneuve	Dune
Gerwig	Lady Bird, Barbie

Source B: Genres and Awards

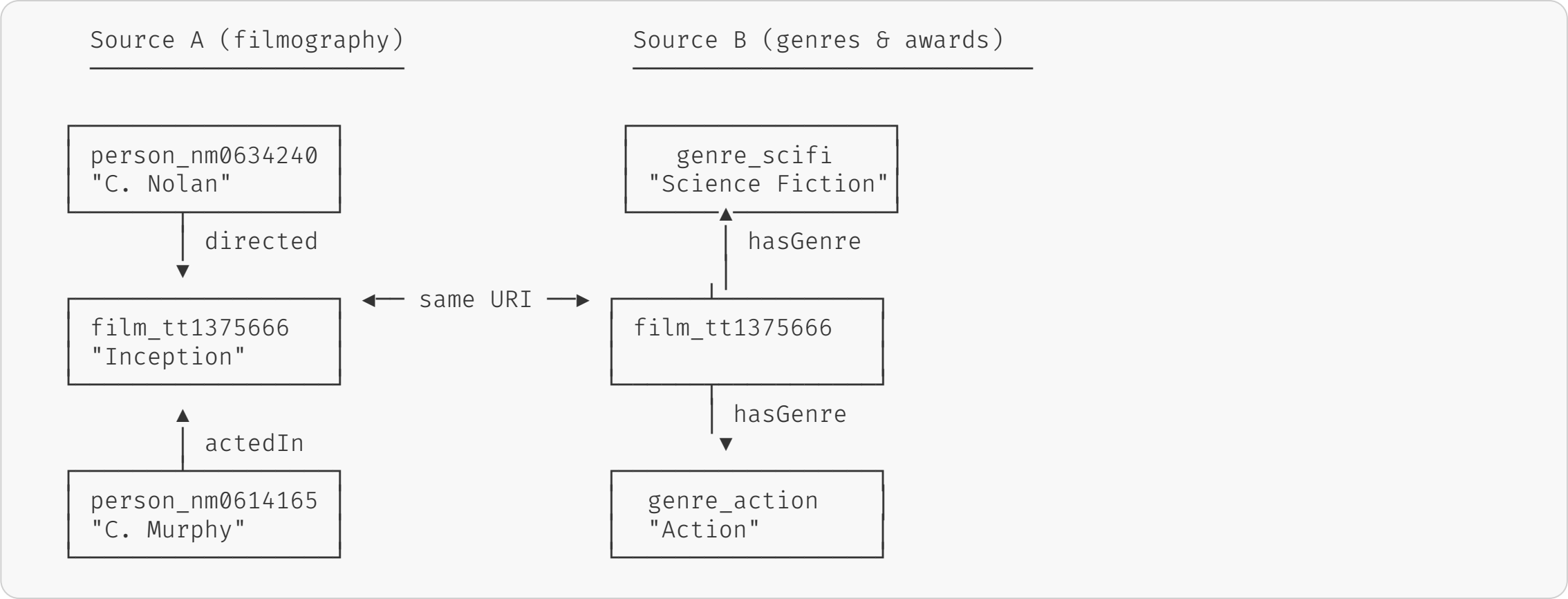
Source B — genre and award data (genres, awards, hasGenre/wonAward edges):

```
@prefix rdf:  <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix ex:   <http://example.org/movie#> .

ex:genre_scifi       rdf:type     ex:Genre ;
                     ex:hasName   "Science Fiction" .

ex:genre_action      rdf:type     ex:Genre ;
                     ex:hasName   "Action" .

ex:film_tt1375666    ex:hasGenre  ex:genre_scifi .
ex:film_tt1375666    ex:hasGenre  ex:genre_action .

ex:award_bestpicture    rdf:type     ex:Award ;
                        ex:hasName   "Academy Award for Best Picture" .

ex:award_bestdirector   rdf:type     ex:Award ;
                        ex:hasName   "Academy Award for Best Director" .

ex:award_bestactor      rdf:type     ex:Award ;
                        ex:hasName   "Academy Award for Best Actor" .

ex:film_tt15398776      ex:wonAward  ex:award_bestpicture .
ex:film_tt15398776      ex:wonAward  ex:award_bestdirector .
ex:film_tt15398776      ex:wonAward  ex:award_bestactor .
```

Source B's triples use the **same film URIs** as Source A. Same URI → same node → edges connect. No mapping table. No ETL.

Film	Genres
Inception	Sci-Fi, Action
Interstellar	Sci-Fi, Drama
Oppenheimer	Drama, Biography
The Dark Knight	Action, Crime, Drama
Dune	Sci-Fi, Action, Drama
Lady Bird	Comedy, Drama
Barbie	Comedy
Batman Begins	*(none in Source B)*
The Dark Knight Rises	*(none in Source B)*

The Structural Merge



The film URI `ex:film_tt1375666` is the integration point — edges from different sources meet at the same node. A traversal from Nolan → Inception → Science Fiction **crosses the source boundary** at this URI.

Tracing "What Genres Appear in Nolan's Films?"

Predict: Nolan directed 6 films in Source A. How many of those 6 films will have `hasGenre` edges in Source B? Not all of them appear there.

The Cross-Source Traversal

Start at Nolan's URI. Follow `directed` edges (Source A) → follow `hasGenre` edges (Source B):

```
ex:person_nm0634240
  ├─directed─> ex:film_tt1375666  —hasGenre—> genre_scifi, genre_action
  ├─directed─> ex:film_tt0816692  —hasGenre—> genre_scifi, genre_drama
  ├─directed─> ex:film_tt15398776 —hasGenre—> genre_drama, genre_biography
  ├─directed─> ex:film_tt0468569  —hasGenre—> genre_action, genre_crime,
  │                                            genre_drama
  ├─directed─> ex:film_tt0372784  (no genre edges in Source B)
  └─directed─> ex:film_tt1345836  (no genre edges in Source B)
```

4 of 6 films have genre data. Under the open-world assumption, the missing genres for Batman Begins and The Dark Knight Rises are **unknown**, not absent.

Result: Science Fiction, Action, Drama, Biography, Crime — spanning two sources, no mapping table.

Validating a Triple

Take `ex:person_nm0634240 ex:directed ex:film_tt1375666`. Check against the property card:

- **Domain:** `ex:Person`. Nolan is `ex:Director`, which is `rdfs:subClassOf ex:Person` — **valid** (subclass satisfies superclass)
- **Range:** `ex:Film`. Inception is `ex:Film` — **valid**
- **Result:** Triple passes

Resolving display names requires knowing the entity's type:

- Films → use `ex:hasTitle`
- Everything else → use `ex:hasName`

This per-type lookup is a real limitation. Topic 9 resolves it with `rdfs:label` as a universal display-name property.

The Six-Step Procedure

Step	Action	Skill
1	**Read the ontology** — fill in property cards, identify class hierarchies	Contract reading (from Topic 7)
2	**Read the data sources** — count entities, identify shared URIs (integration points)	Inventory
3	**Trace traversals by hand** — follow edges, resolve display names, record source of each edge	Cross-source integration
4	**Validate triples** — check subject type vs domain, object type vs range	Structural verification
5	**Convert XML → RDF** — URI vs literal, nesting vs edges, associative entities	Model bridging
6	**Verify the chain** — remove shared URI → traversal breaks; restore → it works	Integration proof

For each step, predict the outcome before checking — same discipline as Topics 4 and 7.

Three Models Side by Side

Dimension	JSON / MongoDB (Topic 6)	XML / XSD (Topic 7)	RDF (Topic 8)
Atomic unit	Document (aggregate)	Element (tree node)	Triple (S, P, O)
Identity scope	`_id` (one collection)	Namespace + element	URI (global, any source)
Schema posture	Schema-on-read	Contract-first (XSD)	Ontology describes; nothing enforces
World assumption	Closed	Closed (validated)	Open (absence = unknown)
Structural variation	Different fields per document	Must be declared in XSD	Different sources, different triples — no schema agreement needed
Cross-source integration	Aggregate-internal; cross-doc requires app logic	Validated document exchange	Structural: same URI = same node
Relationship traversal	Dot notation, `$lookup` (costly)	XPath (within document tree)	Follow edges across sources
Ordering guarantee	Array order preserved	`xs:sequence` enforces sibling order	No ordering — triples are an unordered set
Constraint enforcement	Opt-in `$jsonSchema`	`xmllint --schema` rejects	Ontology describes only; triple store accepts anything
Primary workload	Aggregate read/write	Cross-boundary exchange	Cross-source integration

No model is universally better. The choice depends on the workload.

Check 1: URI Misalignment

A hospital integrates patient data from two systems. System A uses `ex:patient_MRN00412` for a patient. System B uses `ex:pat_412` for the same patient.

A clinician queries "all lab results for patient MRN00412." The query follows edges from `ex:patient_MRN00412`, but System B's lab results point to `ex:pat_412`.

> **What happens?** The graph has two disconnected nodes for the same patient — one with demographic data, one with lab results. The query returns nothing from System B. The fix is an equivalence triple (`owl:sameAs`), but that's manual reconciliation — exactly the mapping-table problem URIs were supposed to eliminate. URI *alignment* upfront is always cheaper than URI *reconciliation* after the fact.

Check 2: Literal vs URI

An e-commerce system stores product categories. Some products use the URI `ex:category_electronics`; one product stores the literal `"Electronics"` instead.

A query asks: "all products in category `ex:category_electronics`."

> **What happens?** The product with `"Electronics"` is missing from results. The literal and the URI are different nodes — the literal has no outgoing edges and does not match the URI. To a human, they look the same. To the graph, they are entirely different nodes.

Check 3: Ontology Catches Structure, Not Facts

A music knowledge graph declares `ex:composed` with domain `ex:Composer` and range `ex:Piece`. Someone adds:

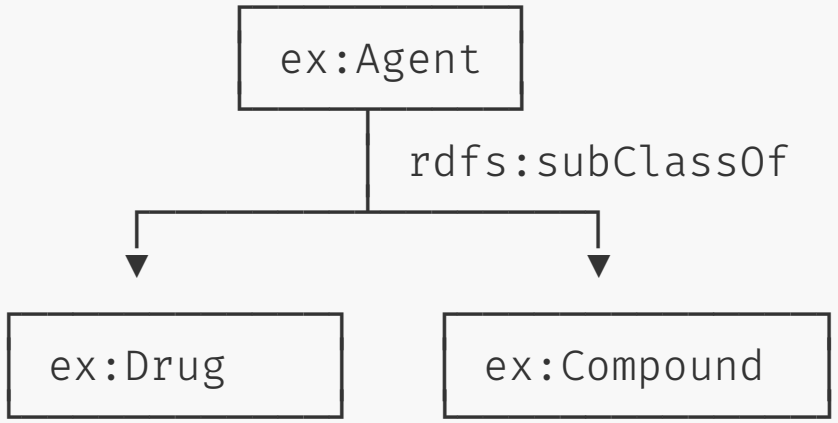
```
ex:person_mozart ex:composed ex:piece_moonlight_sonata
```

Mozart did not compose the Moonlight Sonata (Beethoven did). Mozart is a Composer; Moonlight Sonata is a Piece. Does the ontology reject this triple?

> **No.** The triple passes domain/range validation — valid domain, valid range. The ontology is a structural contract, not a truth-checker. Same limitation as XSD — it catches wrong types, not wrong facts with correct types.

Check 4: Subclass Satisfies Superclass

A pharmaceutical knowledge graph has this class hierarchy:



The property `ex:treats` has domain `ex:Agent`. A researcher writes:
`ex:drug_aspirin ex:treats ex:condition_headache`. Is this a domain violation?

> **No.** Drug is a subclass of Agent. A Drug in an Agent domain position satisfies the constraint — that's what the class hierarchy is for. Do not flag this as a domain violation.

Check 5: Conversion Loses Ordering

A ride-sharing platform converts route data from XML to RDF. The XML preserves the order of waypoints via `xs:sequence`:

```
<route>
  <waypoint>Airport</waypoint>
  <waypoint>Downtown</waypoint>
  <waypoint>University</waypoint>
</route>
```

After conversion, the three waypoints become unordered `hasWaypoint` triples.

> **What is lost?** Waypoint order — the route Airport → Downtown → University is indistinguishable from University → Airport → Downtown. In route planning, order matters. Additional properties (`ex:position "1"^^xsd:integer`) would be needed to preserve sequence. **What is gained?** Each waypoint is now a URI — globally addressable. A second source can add triples about "Downtown" (traffic data, transit connections) without modifying the route source.

Check 6: When RDF Is the Wrong Choice

A tax authority needs to process 50,000 returns per day. Each return must be validated, stored, and auditable. Failed returns must never enter the database. Which enforcement position fits?

> **Engine rejects** (Topic 5). The workload demands transactional guarantees (ACID), strict validation at write time, and auditability. RDF's "nothing rejects" posture would silently absorb invalid returns. The relational model's engine-rejects position — where the DDL constraints block invalid data at write time — matches this workload. RDF is the wrong choice when the workload demands transactional guarantees, aggregate reads, or when URI alignment is absent.

Earlier We Asked: What Kind of Identifier?

In Slide 7, we asked: what kind of identifier could two independent teams adopt *without* coordinating their internal schemas — and still have their data merge automatically?

The answer: **URIs**. Two teams that both use `ex:film_tt1375666` for Inception have their triples merge at that node with zero manual effort. We demonstrated this in the Nolan traversal — `directed` edges from Source A and `hasGenre` edges from Source B met at the same film URI. No mapping table. No ETL. No coordination beyond agreeing on the URI pattern.

The structural guarantee: same URI → same node → edges connect.

Revisiting the Three Failures

Failure	Diagnosis	Fix
Strings vs things	Display names used as identifiers (§8.3)	Assign each film a URI; store the title as a separate literal triple via `ex:hasTitle`
No shared identifiers	Three local ID schemes with no overlap (§8.5)	Adopt a shared URI pattern (IMDB `tt` codes); structural merge eliminates the mapping table
No vocabulary contract	Three predicates for the same relationship (§8.4)	Publish an ontology specifying each property's domain, range, and direction

Every fix maps to a capability we built: URIs for identity, triples for facts, ontology for vocabulary. The failures are now diagnosable with precision.

Concepts Planted Here That Recur

Concept	Introduced here as	Recurs in
URI as global identity	IMDB-based URIs for films/persons	Topic 9 (SPARQL variables bind to URIs), Topic 11 (cross-model identity layer)
Nothing rejects	Ontology describes but does not enforce	Topic 10 (property graph: schema-optional), Topic 11 (enforcement strategy per workload)
Property card	Template for reading ontology predicates	Topic 9 (property cards inform SPARQL pattern design)
Strings vs things	Literal = dead end; URI = traversable	Topic 9 (FILTER on literals vs pattern-match on URIs), Topic 11 (identifier strategy)
Open-world assumption	Absence = unknown, not false	Topic 9 (OPTIONAL patterns), Topic 11 (contrast with closed-world relational)
Associative entity (RDF)	Performance intermediate node	Topic 10 (edge properties as simpler alternative)

Course Map

```
Part 1 — Relational Foundation
├── Ch 1  Introduction to Data Models
├── Ch 2  Conceptual Modelling (ERD)
├── Ch 3  Normalisation (BCNF & Star Schemas)
├── Ch 4  SQL Querying (FWGHOS Pipeline)
└── Ch 5  Enforcement (Constraints, Access Control, Transactions, Indexes)

Part 2 — Alternative Models
├── Ch 6  Document Model (JSON & MongoDB)
├── Ch 7  XML for Contracts (XSD & XPath)
├── Ch 8  RDF and Knowledge Graphs ◄── you are here
├── Ch 9  SPARQL Querying
└── Ch 10 Property Graphs (Cypher)

Part 3 — Capstone
└── Ch 11 Multi-Model Design
```

Topic 8 fills the "nothing rejects" position on the enforcement spectrum and introduces the global identity layer (URIs) that Topics 9–11 build on. Topic 9 formalises the hand-traced traversals as SPARQL queries. Topic 10 introduces property graphs. Topic 11 assigns each model to a workload.

Lab Preview

Lab 8 — RDF and Knowledge Graphs: Read, Trace, Validate, Convert

A hand-worked knowledge graph exercise — no code, no Colab.

- **Part 1** (~15 min): Read the ontology — fill in property cards for all predicates
- **Part 2** (~10 min): Read the data sources — count entities, identify shared URIs
- **Part 3** (~30 min): Trace traversals by hand — circle URIs, draw lines across sheets
- **Part 4** (~15 min): Validate triples against the ontology
- **Part 5** (~15 min): Convert XML to RDF triples by hand
- **Part 6** (~5 min): Verify the chain — remove a shared URI, confirm traversal breaks

Format: Reasoning and hand-tracing on printed Turtle files. **Total ~90 min.**

File: `08-lab-rdf-and-knowledge-graphs.md`